

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

01:0001 """Atomic artifacts and reproducible identities for application runs."""
01:0002 from __future__ import annotations
01:0004 import hashlib
01:0005 import json
01:0006 import os
01:0007 import tempfile
01:0008 from pathlib import Path
01:0011 def fingerprint(value: object) -> str:
01:0012     payload = json.dumps(value, sort_keys=True, separators=(",", ":"), allow_nan=False)
01:0013     return hashlib.sha256(payload.encode()).hexdigest()
01:0016 def file_hash(path: Path) -> str:
01:0017     digest = hashlib.sha256()
01:0018     with path.open("rb") as stream:
01:0019         for block in iter(lambda: stream.read(1024 * 1024), b''):
01:0020             digest.update(block)
01:0021     return digest.hexdigest()
01:0024 def file_identity(path: Path, content_hash: bool = False) -> dict:
01:0025     path = path.resolve(strict=True)
01:0026     stat = path.stat()
01:0027     result = {"path": str(path), "bytes": stat.st_size, "mtime_ns": stat.st_mtime_ns}
01:0028     if content_hash:
01:0029         result["sha256"] = file_hash(path)
01:0030     return result
01:0033 def atomic_text(path: Path, text: str) -> None:
01:0034     path.parent.mkdir(parents=True, exist_ok=True)
01:0035     fd, temporary = tempfile.mkstemp(prefix=f"{path.name}.", dir=path.parent)
01:0036     try:
01:0037         with os.fdopen(fd, "w", encoding="utf-8") as stream:
01:0038             stream.write(text)
01:0039             stream.flush()
01:0040             os.fsync(stream.fileno())
01:0041             os.replace(temporary, path)
01:0042     finally:
01:0043         Path(temporary).unlink(missing_ok=True)
01:0046 def save_json(path: Path, value: object) -> None:
01:0047     atomic_text(path, json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + "\n")
01:0050 def load_json(path: Path):
01:0051     return json.loads(path.read_text(encoding="utf-8"))
02:0001 """Preflight images and videos without loading a neural network."""
02:0002 from __future__ import annotations
02:0004 from pathlib import Path
02:0005 import re
02:0007 import cv2
02:0008 import numpy as np
02:0010 from .storage import file_identity, fingerprint
02:0012 IMAGES = {".jpg", ".jpeg", ".png", ".bmp", ".webp", ".tif", ".tiff"}
02:0013 VIDEOS = {".mp4", ".avi", ".mov", ".mkv", ".m4v", ".webm"}
02:0016 def natural_key(path: Path):
02:0017     return tuple((0, int(x)) if x.isdigit() else (1, x.lower()))
02:0018                 for x in re.split(r"(\d+)", str(path)))

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

02:0021 def scan_media(source: Path, recursive: bool = False, hash_inputs: bool = False) -> dict:
02:0022     source = source.resolve(strict=True)
02:0023     candidates = ([source] if source.is_file() else
02:0024                   sorted(source.rglob("*") if recursive else source.iterdir(), key=natural_key))
02:0025     records = []
02:0026     ignored = 0
02:0027     for path in candidates:
02:0028         if not path.is_file():
02:0029             continue
02:0030         suffix = path.suffix.lower()
02:0031         if suffix not in IMAGES | VIDEOS:
02:0032             ignored += 1
02:0033             continue
02:0034         record = {"identity": file_identity(path, hash_inputs), "kind": "image" if suffix in IMAGES
else "video"}
02:0035         record["id"] = fingerprint(record["identity"])[20]
02:0036         if record["kind"] == "image":
02:0037             image = cv2.imread(str(path))
02:0038             record["readable"] = image is not None
02:0039             if image is not None:
02:0040                 record.update(width=int(image.shape[1]), height=int(image.shape[0]), frames=1)
02:0041         else:
02:0042             capture = cv2.VideoCapture(str(path))
02:0043             try:
02:0044                 ok, image = capture.read()
02:0045                 fps = float(capture.get(cv2.CAP_PROP_FPS))
02:0046                 frames = float(capture.get(cv2.CAP_PROP_FRAME_COUNT))
02:0047                 record["readable"] = bool(ok and np.isfinite(fps) and fps > 0)
02:0048                 if ok:
02:0049                     record.update(width=int(image.shape[1]), height=int(image.shape[0]),
02:0050                                   fps=fps if np.isfinite(fps) else None,
02:0051                                   frames=int(frames) if np.isfinite(frames) and frames > 0 else
None)
02:0052             finally:
02:0053                 capture.release()
02:0054             if not record["readable"]:
02:0055                 record["error"] = "Cannot decode media or determine a positive video frame rate"
02:0056             records.append(record)
02:0057         return {"schema": "egohand3d.media/1", "source": str(source), "recursive": recursive,
02:0058               "files": records, "summary": {"files": len(records), "ignored_files": ignored,
02:0059               "readable": sum(r["readable"] for r in records),
02:0060               "unreadable": sum(not r["readable"] for r in records)}}
02:0063 def iter_frames(manifest: dict, stride: int = 1, limit: int = 0, sequence_fps: float | None =
None):
02:0064     """Yield decoded images and metadata; video timestamps assume constant frame rate.
02:0065     A decode failure is yielded rather than silently counted as a success. For an
02:0066     image sequence, users must explicitly supply its frame rate.
02:0067     """
02:0068
02:0069     if stride < 1 or limit < 0 or (sequence_fps is not None and sequence_fps <= 0):
02:0070         raise ValueError("stride/fps must be positive and limit must be nonnegative")
02:0071     emitted = 0
02:0072     image_index = 0
02:0073     for item in manifest["files"]:

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

02:0074     path = Path(item["identity"]["path"])
02:0075     identity = file_identity(path, "sha256" in item["identity"])
02:0076     if identity != item["identity"]:
02:0077         raise ValueError(f"Input changed after preflight: {path}")
02:0078     common = {"source_path": str(path), "media_id": item["id"], "image": path.name}
02:0079     if item["kind"] == "image" or not item["readable"]:
02:0080         metadata = {**common, "sample_id": item["id"], "frame_index": image_index,
02:0081                     "sequence_id": fingerprint(manifest["source"])[20] if sequence_fps else
item["id"],
02:0082                     "timestamp_s": image_index / sequence_fps if sequence_fps else None,
02:0083                     "time_source": "user_frame_rate" if sequence_fps else "none"}
02:0084         image_index += 1
02:0085         yield metadata, cv2.imread(str(path)) if item["readable"] else None
02:0086         emitted += 1
02:0087     else:
02:0088         capture = cv2.VideoCapture(str(path))
02:0089         index = 0
02:0090         try:
02:0091             while not limit or emitted < limit:
02:0092                 ok, image = capture.read()
02:0093                 if not ok:
02:0094                     if item.get("frames") and index < item["frames"]:
02:0095                         yield {**common, "sample_id": f"{item['id']}_{index:08d}",
02:0096                               "sequence_id": item["id"], "frame_index": index,
02:0097                               "timestamp_s": index / item["fps"], "time_source":
"container_fps",
02:0098                               "decode_error": "Video ended before reported frame count"}, None
02:0099                     break
02:0100                 if index % stride == 0:
02:0101                     yield {**common, "sample_id": f"{item['id']}_{index:08d}",
02:0102                             "sequence_id": item["id"], "frame_index": index,
02:0103                             "timestamp_s": index / item["fps"], "time_source": "container_fps"},
image
02:0104                     emitted += 1
02:0105                     index += 1
02:0106                 finally:
02:0107                     capture.release()
02:0108             if limit and emitted >= limit:
02:0109                 return
03:0001     """Persistent, single-writer processing runs with cooperative cancellation."""
03:0002     from __future__ import annotations
03:0004     from contextlib import contextmanager
03:0005     from datetime import datetime, timezone
03:0006     from pathlib import Path
03:0007     import fcntl
03:0008     import json
03:0009     import sqlite3
03:0011     from .storage import file_hash, fingerprint
03:0014     def now() -> str:
03:0015         return datetime.now(timezone.utc).isoformat()
03:0018     @contextmanager
03:0019     def run_lock(directory: Path):
03:0020         directory.mkdir(parents=True, exist_ok=True)

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

03:0021         with (directory / ".run.lock").open("a") as handle:
03:0022             try:
03:0023                 fcntl.flock(handle, fcntl.LOCK_EX | fcntl.LOCK_NB)
03:0024             except BlockingIOError as exc:
03:0025                 raise RuntimeError("Another process owns this output directory") from exc
03:0026             try:
03:0027                 yield
03:0028             finally:
03:0029                 fcntl.flock(handle, fcntl.LOCK_UN)
03:0032 class JobStore:
03:0033     def __init__(self, path: Path):
03:0034         self.path = path
03:0035         self.db = sqlite3.connect(path, timeout=15)
03:0036         self.db.row_factory = sqlite3.Row
03:0037         self.db.execute("PRAGMA journal_mode=WAL")
03:0038         self.db.executescript("""
03:0039             CREATE TABLE IF NOT EXISTS metadata (key TEXT PRIMARY KEY, value TEXT NOT NULL);
03:0040             CREATE TABLE IF NOT EXISTS items (
03:0041                 id TEXT PRIMARY KEY, source TEXT NOT NULL, state TEXT NOT NULL,
03:0042                 attempts INTEGER NOT NULL DEFAULT 0, error TEXT, record TEXT,
03:0043                 artifacts TEXT, seconds REAL, updated TEXT NOT NULL
03:0044             );
03:0045             """)
03:0047     def close(self):
03:0048         self.db.close()
03:0050     def get(self, key: str, default=None):
03:0051         row = self.db.execute("SELECT value FROM metadata WHERE key=?", (key,)).fetchone()
03:0052         return json.loads(row[0]) if row else default
03:0054     def set(self, key: str, value):
03:0055         with self.db:
03:0056             self.db.execute("INSERT OR REPLACE INTO metadata VALUES (?,?)", (key,
json.dumps(value)))
03:0058     def initialize(self, config: dict, resume: bool):
03:0059         previous = self.get("config_hash")
03:0060         if previous:
03:0061             if not resume:
03:0062                 raise ValueError("Run already exists; use --resume or a different output
directory")
03:0063             if previous != fingerprint(config):
03:0064                 raise ValueError("Inputs, code, assets or options changed; use a new output
directory")
03:0065         else:
03:0066             self.set("config_hash", fingerprint(config))
03:0067             self.set("config", config)
03:0068             self.set("created", now())
03:0069             self.set("cancel_requested", False)
03:0070             self.set("state", "running")
03:0071             with self.db:
03:0072                 self.db.execute("UPDATE items SET state='pending' WHERE state='running'")
03:0074     def completed(self, sample_id: str, directory: Path) -> bool:
03:0075         row = self.db.execute("SELECT state, artifacts FROM items WHERE id=?",
(sample_id,)).fetchone()
03:0076         if row is None or row["state"] != "completed":
03:0077             return False

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

03:0078         artifacts = json.loads(row["artifacts"] or "{}")
03:0079         return bool(artifacts) and all((directory / name).is_file() and file_hash(directory / name)
== sha
03:0080                                     for name, sha in artifacts.items())
03:0082         def begin(self, sample_id: str, source: str):
03:0083             with self.db:
03:0084                 self.db.execute("""INSERT INTO items(id,source,state,attempts,updated) VALUES
(?,?, 'running',1,?)
03:0085                 ON CONFLICT(id) DO UPDATE SET
state='running',attempts=attempts+1,error=NULL,updated=excluded.updated""",
03:0086                 (sample_id, source, now()))
03:0088         def finish(self, sample_id: str, seconds: float, record: str, artifacts: dict):
03:0089             with self.db:
03:0090                 self.db.execute("""UPDATE items SET
state='completed',seconds=?,record=?,artifacts=?,error=NULL,
03:0091                 updated=? WHERE id=?""", (seconds, record, json.dumps(artifacts), now(),
sample_id))
03:0093         def fail(self, sample_id: str, error: str):
03:0094             with self.db:
03:0095                 self.db.execute("UPDATE items SET state='failed',error=?,updated=? WHERE id=?", (error,
now(), sample_id))
03:0097         def records(self):
03:0098             return self.db.execute("SELECT * FROM items ORDER BY rowid").fetchall()
03:0100         def summary(self) -> dict:
03:0101             counts = dict(self.db.execute("SELECT state,COUNT(*) FROM items GROUP BY state"))
03:0102             return {"state": self.get("state"), "created": self.get("created"),
"updated": self.get("updated"), "cancel_requested": self.get("cancel_requested",
False),
03:0104                 "counts": counts, "discovered_items": sum(counts.values())}
03:0107         def inspect_jobs(directory: Path, cancel: bool = False) -> dict:
03:0108             path = directory / "tasks.sqlite"
03:0109             if not path.is_file():
03:0110                 raise FileNotFoundError(f"No run database at {path}")
03:0111             store = JobStore(path)
03:0112             try:
03:0113                 if cancel:
03:0114                     store.set("cancel_requested", True)
03:0115                 return store.summary()
03:0116             finally:
03:0117                 store.close()
04:0001 """Explicit MANO interchange; WiLoR left hands use a reflected RIGHT model.
04:0003 Rotations remain in the canonical right MANO model. Reflection is applied to
04:0004 decoded geometry before camera translation; these are not left-model poses.
04:0005 """
04:0006 from __future__ import annotations
04:0008 import copy
04:0009 import numpy as np
04:0010 from scipy.spatial.transform import Rotation
04:0012 SCHEMA = "egohand3d.mano/1"
04:0015 def rotations(value, count: int = 16) -> np.ndarray:
04:0016     value = np.asarray(value, dtype=np.float64)
04:0017     if value.shape != (count, 3, 3) or not np.isfinite(value).all():
04:0018         raise ValueError(f"Expected {count} finite 3x3 rotation matrices")
04:0019     if not np.allclose(value @ value.swapaxes(-1, -2), np.eye(3), atol=2e-4):
04:0020         raise ValueError("Rotation matrices are not orthonormal")
04:0021     if not np.allclose(np.linalg.det(value), 1, atol=2e-4):
04:0022         raise ValueError("Rotations must have determinant +1; reflection is separate")

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

04:0023     return value
04:0026 def encode_hand(hand: dict) -> dict:
04:0027     params = hand["pred_mano_params"]
04:0028     arrays = {}
04:0029     for name in ["global_orient", "hand_pose", "betas"]:
04:0030         payload = params[name]
04:0031         array = np.asarray(payload["values"], dtype=np.float64)
04:0032         if list(array.shape) != payload["shape"]:
04:0033             raise ValueError(f"MANO {name}: declared shape differs from values")
04:0034         arrays[name] = array
04:0035     if arrays["global_orient"].shape != (1, 3, 3) or arrays["hand_pose"].shape != (15, 3, 3):
04:0036         raise ValueError("Runtime MANO must supply rotation matrices, not inferred axis angles")
04:0037     pose = rotations(np.concatenate([arrays["global_orient"], arrays["hand_pose"]]))
04:0038     width, height = hand["img_size_wh"]
04:0039     focal = float(hand["focal_length_px"])
04:0040     packet = {
04:0041         "schema": SCHEMA, "model_hand": "right", "hand_side": hand["hand_side"],
04:0042         "mirror_model_x": hand["hand_side"] == "left", "length_unit": "m", "angle_unit": "rad",
04:0043         "camera_axes": "x_right_y_down_z_forward", "pose_space": "canonical_right_mano",
04:0044         "geometry_origin": "MANO native origin; not explicitly wrist-centered",
04:0045         "pose_rotmat": pose.tolist(), "pose_axis_angle":
Rotation.from_matrix(pose).as_rotvec().tolist(),
04:0046         "betas": arrays["betas"].tolist(), "translation_camera": hand["cam_t_full"],
04:0047         "intrinsics": [[focal, 0., width / 2], [0., focal, height / 2], [0., 0., 1.]],
04:0048         "intrinsics_source": "WiLoR assumed focal length; not a camera calibration",
04:0049         "image_size_wh": [width, height],
04:0050     }
04:0051     validate_packet(packet)
04:0052     return packet
04:0055 def validate_packet(packet: dict) -> None:
04:0056     if packet.get("schema") != SCHEMA or packet.get("model_hand") != "right":
04:0057         raise ValueError("Unsupported MANO schema/model hand")
04:0058     if packet.get("hand_side") not in {"left", "right"}:
04:0059         raise ValueError("hand_side must be left or right")
04:0060     if packet.get("mirror_model_x") is not (packet["hand_side"] == "left"):
04:0061         raise ValueError("Hand side and reflection disagree")
04:0062     if packet.get("length_unit") != "m" or packet.get("angle_unit") != "rad":
04:0063         raise ValueError("Interchange uses meters and radians")
04:0064     pose = rotations(packet["pose_rotmat"])
04:0065     aa = np.asarray(packet["pose_axis_angle"], dtype=np.float64)
04:0066     if aa.shape != (16, 3) or not np.isfinite(aa).all():
04:0067         raise ValueError("Expected 16 finite axis-angle vectors")
04:0068     if not np.allclose(Rotation.from_rotvec(aa).as_matrix(), pose, atol=2e-4):
04:0069         raise ValueError("Axis angles and rotation matrices describe different poses")
04:0070     for key, shape in [("betas", (10,)), ("translation_camera", (3,)), ("intrinsics", (3, 3)),
04:0071         ("image_size_wh", (2,))]:
04:0072         array = np.asarray(packet[key], dtype=float)
04:0073         if array.shape != shape or not np.isfinite(array).all():
04:0074             raise ValueError(f"Invalid {key}; expected finite shape {shape}")
04:0075     k = np.asarray(packet["intrinsics"])
04:0076     if k[0, 0] <= 0 or k[1, 1] <= 0 or not np.allclose(k[2], [0, 0, 1]):

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

04:0077         raise ValueError("Invalid pinhole intrinsics")
04:0078     if min(packet["image_size_wh"]) <= 0:
04:0079         raise ValueError("Image dimensions must be positive")
04:0082 def project(points_camera, intrinsics) -> tuple[np.ndarray, np.ndarray]:
04:0083     points = np.asarray(points_camera, dtype=np.float64)
04:0084     valid = np.isfinite(points).all(axis=-1) & (points[:, 2] > 1e-8)
04:0085     uv = np.zeros((len(points), 2), dtype=np.float64)
04:0086     homogeneous = points[valid] @ np.asarray(intrinsics, dtype=float).T
04:0087     uv[valid] = homogeneous[:, :2] / homogeneous[:, 2:]
04:0088     return uv, valid
04:0091 class ManoDecoder:
04:0092     def __init__(self, layer):
04:0093         self.layer = layer
04:0095     @classmethod
04:0096     def from_directory(cls, directory, device: str = "cpu"):
04:0097         # Use the same declared 21-joint mapping as the external runtime.
04:0098         from wilor.models.mano_wrapper import MANO
04:0099         return cls(MANO(model_path=str(directory), is_rhand=True, use_pca=False,
04:0100                        flat_hand_mean=True).to(device).eval())
04:0102     def decode(self, packet: dict) -> dict:
04:0103         import torch
04:0104         validate_packet(packet)
04:0105         device = next(self.layer.buffers()).device
04:0106         pose = torch.as_tensor(packet["pose_rotmat"], dtype=torch.float32, device=device)[None]
04:0107         betas = torch.as_tensor(packet["betas"], dtype=torch.float32, device=device)[None]
04:0108         previous_tf32 = torch.backends.cuda.matmul.allow_tf32
04:0109         try:
04:0110             torch.backends.cuda.matmul.allow_tf32 = False
04:0111             with torch.inference_mode():
04:0112                 output = self.layer(global_orient=pose[:, :1], hand_pose=pose[:, 1:], betas=betas,
pose2rot=False)
04:0113             finally:
04:0114                 torch.backends.cuda.matmul.allow_tf32 = previous_tf32
04:0115             joints = output.joints[0].cpu().numpy().copy()
04:0116             vertices = output.vertices[0].cpu().numpy().copy()
04:0117             if packet["mirror_model_x"]:
04:0118                 joints[:, 0] *= -1
04:0119                 vertices[:, 0] *= -1
04:0120             translation = np.asarray(packet["translation_camera"])
04:0121             uv, visible_depth = project(joints + translation, packet["intrinsics"])
04:0122             return {"joints_model": joints, "vertices_model": vertices,
04:0123                    "joints_camera": joints + translation, "vertices_camera": vertices + translation,
04:0124                    "joints_2d": uv, "positive_depth": visible_depth}
04:0126     def check(self, packet: dict, hand: dict, tolerance_m: float = 1e-5, tolerance_px: float =
0.05) -> dict:
04:0127         decoded = self.decode(packet)
04:0128         joint_error = np.linalg.norm(decoded["joints_model"] - np.asarray(hand["joints_3d_root"]),
axis=1)
04:0129         vertex_error = np.linalg.norm(decoded["vertices_model"] -
np.asarray(hand["vertices_3d_root"]), axis=1)
04:0130         valid = decoded["positive_depth"]
04:0131         uv_error = np.linalg.norm(decoded["joints_2d"][valid] -
np.asarray(hand["joints_2d_xy"][valid], axis=1)
04:0132         maximum_px = float(uv_error.max()) if valid.any() else None
04:0133         return {"max_joint_error_m": float(joint_error.max()), "max_vertex_error_m":
float(vertex_error.max()),

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

04:0134         "max_reprojection_error_px": maximum_px, "num_projectable_joints":
            int(valid.sum()),
04:0135         "tolerance_m": tolerance_m, "tolerance_px": tolerance_px,
04:0136         "passed": bool(joint_error.max() <= tolerance_m and vertex_error.max() <=
            tolerance_m
04:0137                     and maximum_px is not None and maximum_px <= tolerance_px)}
04:0140 def interpolate_packets(first: dict, second: dict, weight: float) -> dict:
04:0141     """Geodesic interpolation of each rotation, linear shape and translation."""
04:0142     validate_packet(first)
04:0143     validate_packet(second)
04:0144     if first["hand_side"] != second["hand_side"] or not 0 <= weight <= 1:
04:0145         raise ValueError("Interpolation needs the same hand side and weight in [0,1]")
04:0146     if first["image_size_wh"] != second["image_size_wh"] or not np.allclose(first["intrinsics"],
            second["intrinsics"]):
04:0147         raise ValueError("Interpolation needs matching camera metadata")
04:0148     a, b = np.asarray(first["pose_rotmat"]), np.asarray(second["pose_rotmat"])
04:0149     relative = a.swapaxes(-1, -2) @ b
04:0150     delta = Rotation.from_matrix(relative).as_rotvec() * weight
04:0151     pose = a @ Rotation.from_rotvec(delta).as_matrix()
04:0152     out = copy.deepcopy(second)
04:0153     out["pose_rotmat"] = pose.tolist()
04:0154     out["pose_axis_angle"] = Rotation.from_matrix(pose).as_rotvec().tolist()
04:0155     for key in ["betas", "translation_camera"]:
04:0156         out[key] = ((1 - weight) * np.asarray(first[key]) + weight *
            np.asarray(second[key])).tolist()
04:0157     return out
05:0001 """Transparent diagnostics, not learned joint confidence or proof of accuracy."""
05:0002 from __future__ import annotations
05:0004 import numpy as np
05:0006 from .visualization import HAND_CONNECTIONS
05:0009 def inspect_hand(hand: dict, min_detection_score: float = 0.5) -> dict:
05:0010     uv = np.asarray(hand["joints_2d_xy"], dtype=float)
05:0011     xyz = np.asarray(hand["joints_3d_camera"], dtype=float)
05:0012     width, height = hand["img_size_wh"]
05:0013     finite = np.isfinite(uv).all(axis=1) & np.isfinite(xyz).all(axis=1)
05:0014     front = xyz[:, 2] > 1e-8
05:0015     inside = (uv[:, 0] >= 0) & (uv[:, 0] < width) & (uv[:, 1] >= 0) & (uv[:, 1] < height)
05:0016     lengths = np.array([np.linalg.norm(xyz[a] - xyz[b]) for a, b in HAND_CONNECTIONS])
05:0017     flags = []
05:0018     if not finite.all():
05:0019         flags.append("nonfinite_geometry")
05:0020     if not front.all():
05:0021         flags.append("nonpositive_depth")
05:0022     if not inside.all():
05:0023         flags.append("joints_outside_image")
05:0024     if hand["bbox_score"] < min_detection_score:
05:0025         flags.append("low_detection_score")
05:0026     if np.any(lengths < 1e-5):
05:0027         flags.append("degenerate_bone")
05:0028     closure = hand.get("roundtrip")
05:0029     if closure and not closure["passed"]:
05:0030         flags.append("mano_roundtrip_failed")
05:0031     return {"flags": flags, "num_finite_joints": int(finite.sum()),
05:0032           "num_positive_depth": int(front.sum()), "num_inside_image": int(inside.sum()),

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

05:0033         "num_usable_joints": int((finite & front & inside).sum()),
05:0034         "detection_score": float(hand["bbox_score"]),
05:0035         "score_note": "Detector score is not per-joint confidence",
05:0036         "bone_lengths_m": [float(v) if np.isfinite(v) else None for v in lengths],
05:0037         "thresholds": {"min_detection_score": min_detection_score, "min_bone_length_m": 1e-5}}
06:0001 """One inference per frame; all exports share the same predictions."""
06:0002 from __future__ import annotations
06:0004 import csv
06:0005 import io
06:0006 import platform
06:0007 import signal
06:0008 import sys
06:0009 import time
06:0010 from collections import Counter
06:0011 from pathlib import Path
06:0013 import cv2
06:0014 import numpy as np
06:0016 from .jobs import JobStore, now, run_lock
06:0017 from .mano_codec import ManoDecoder, encode_hand
06:0018 from .media import iter_frames, scan_media
06:0019 from .quality import inspect_hand
06:0020 from .storage import atomic_text, file_hash, file_identity, load_json, save_json
06:0021 from .visualization import draw_record_on_image
06:0023 ROOT = Path(__file__).resolve().parents[1]
06:0026 def code_identity() -> dict:
06:0027     paths = sorted((ROOT / "egohand3d").glob("*.py"))
06:0028     paths += sorted((ROOT / "wilor").rglob("*.py"))
06:0029     return {str(path.relative_to(ROOT)): file_hash(path) for path in paths}
06:0032 def write_obj(path: Path, vertices, faces, mirrored: bool):
06:0033     faces = np.asarray(faces)
06:0034     if mirrored:
06:0035         faces = faces[:, [0, 2, 1]]
06:0036     lines = ["# EgoHand3D camera coordinates, meters"]
06:0037     lines += ["v %.9g %.9g %.9g" % tuple(v) for v in vertices]
06:0038     lines += ["f %d %d %d" % tuple(f + 1) for f in faces]
06:0039     atomic_text(path, "\n".join(lines) + "\n")
06:0042 def export_frame(directory: Path, record: dict, image, faces, overlays: bool, meshes: bool) ->
dict:
06:0043     sample_id = record["sample_id"]
06:0044     files = []
06:0045     path = directory / "records" / f"{sample_id}.json"
06:0046     save_json(path, record)
06:0047     files.append(path)
06:0048     for kind, keys in {
06:0049         "2d": ["joints_2d_xy", "joints_2d_scores"],
06:0050         "3d": ["joints_3d_root", "joints_3d_camera", "cam_t_full"],
06:0051         "mano": ["mano"],
06:0052     }.items():
06:0053         payload = {key: record[key] for key in ["sample_id", "source_path", "sequence_id",
"frame_index", "timestamp_s"]}
06:0054         payload["hands"] = [{key: hand[key] for key in ["hand_index", "hand_side", "bbox_xyxy",
"bbox_score", *keys]}
06:0055                             for hand in record["hands"]]

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

06:0056         output = directory / kind / f"{sample_id}.json"
06:0057         save_json(output, payload)
06:0058         files.append(output)
06:0059     if meshes:
06:0060         for hand in record["hands"]:
06:0061             output = directory / "meshes" / f"{sample_id}_{hand['hand_index']}.obj"
06:0062             write_obj(output, hand["vertices_3d_camera"], faces, hand["hand_side"] == "left")
06:0063             files.append(output)
06:0064     if overlays:
06:0065         output = directory / "overlays" / f"{sample_id}.jpg"
06:0066         output.parent.mkdir(parents=True, exist_ok=True)
06:0067         # Complete-file hashes prevent a partial image being reused after interruption.
06:0068         overlay = draw_record_on_image(image, record, score_threshold=0.01)
06:0069         if not cv2.imwrite(str(output), overlay):
06:0070             raise OSError(f"Cannot write {output}")
06:0071         files.append(output)
06:0072     return {str(path.relative_to(directory)): file_hash(path) for path in files}
06:0075 def write_run_summary(directory: Path, store: JobStore, invocation: dict) -> dict:
06:0076     rows = store.records()
06:0077     successful = [r for r in rows if r["state"] == "completed"]
06:0078     hand_count = 0
06:0079     detected = 0
06:0080     flags = Counter()
06:0081     roundtrip_failed = 0
06:0082     records_index = []
06:0083     for row in successful:
06:0084         record = load_json(directory / row["record"])
06:0085         hand_count += len(record["hands"])
06:0086         detected += bool(record["hands"])
06:0087         for hand in record["hands"]:
06:0088             flags.update(hand["quality"]["flags"])
06:0089             roundtrip_failed += not hand["roundtrip"]["passed"]
06:0090         records_index.append({key: record[key] for key in ["sample_id", "source_path",
"sequence_id", "timestamp_s", "frame_index"]
06:0091                               | {"record": row["record"], "hands": len(record["hands"])})}
06:0092     summary = {**store.summary(), "schema": "egohand3d.run/1", "frames_completed": len(successful),
06:0093               "frames_with_hands": detected, "hands": hand_count,
06:0094               "detection_coverage": detected / len(successful) if successful else None,
06:0095               "coverage_note": "Fraction of processed frames with predictions; not detection
recall without GT",
06:0096               "quality_flags": dict(flags), "roundtrip_failures": roundtrip_failed,
06:0097               "mean_processing_seconds": float(np.mean([r["seconds"] for r in successful])) if
successful else None,
06:0098               "latest_invocation": invocation,
06:0099               "failures": [{"sample_id": r["id"], "error": r["error"]} for r in rows if r["state"]
== "failed"]}
06:0100     save_json(directory / "summary.json", summary)
06:0101     save_json(directory / "index.json", {"schema": "egohand3d.index/1", "records": records_index})
06:0102     buffer = io.StringIO()
06:0103     writer = csv.DictWriter(buffer, fieldnames=["sample_id", "source_path", "sequence_id",
"timestamp_s", "frame_index", "record", "hands"])
06:0104     writer.writeheader()
06:0105     writer.writerows(records_index)
06:0106     atomic_text(directory / "index.csv", buffer.getvalue())
06:0107     return summary

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

06:0110 def process(args) -> dict:
06:0111     directory = Path(args.out).resolve()
06:0112     manifest = scan_media(Path(args.input), args.recursive, args.hash_inputs)
06:0113     if not manifest["files"]:
06:0114         raise ValueError("No supported input media")
06:0115     if args.sequence_fps and any(f["kind"] != "image" for f in manifest["files"]):
06:0116         raise ValueError("--sequence-fps is for a directory of images only")
06:0117     assets = {key: file_identity(Path(getattr(args, key)), args.hash_assets)
06:0118               for key in ["checkpoint", "cfg", "detector"]}
06:0119     for name in ["MANO_RIGHT.pkl", "mano_mean_params.npz"]:
06:0120         assets[name] = file_identity(ROOT / "mano_data" / name, args.hash_assets)
06:0121     config = {"pipeline_version": 1, "manifest": manifest, "assets": assets, "code":
code_identity(),
06:0122              "options": {key: getattr(args, key) for key in ["stride", "limit", "sequence_fps",
"device",
06:0123                      "detector_conf", "rescale_factor", "batch_size", "no_mesh", "no_overlay"]}}
06:0124     with run_lock(directory):
06:0125         store = JobStore(directory / "tasks.sqlite")
06:0126         try:
06:0127             store.initialize(config, args.resume)
06:0128             save_json(directory / "manifest.json", manifest)
06:0129             save_json(directory / "run_config.json", config)
06:0130             return _process(args, directory, store, manifest)
06:0131         finally:
06:0132             store.close()
06:0133 def _process(args, directory: Path, store: JobStore, manifest: dict) -> dict:
06:0134     runtime = None
06:0135     started = time.perf_counter()
06:0136     invocation = {"started": now(), "processed": 0, "reused": 0, "python": sys.version, "platform":
platform.platform()}
06:0137     previous_sigterm = signal.getsignal(signal.SIGTERM)
06:0138     def interrupted(signum, frame):
06:0139         raise KeyboardInterrupt
06:0140     signal.signal(signal.SIGTERM, interrupted)
06:0141     try:
06:0142         for metadata, image in iter_frames(manifest, args.stride, args.limit, args.sequence_fps):
06:0143             if store.get("cancel_requested", False):
06:0144                 store.set("state", "cancelled")
06:0145                 break
06:0146             sample_id = metadata["sample_id"]
06:0147             if store.completed(sample_id, directory):
06:0148                 invocation["reused"] += 1
06:0149                 continue
06:0150             for attempt in range(args.retries + 1):
06:0151                 store.begin(sample_id, metadata["source_path"])
06:0152                 try:
06:0153                     if image is None:
06:0154                         raise ValueError(metadata.get("decode_error", "Unreadable image/video"))
06:0155                     if runtime is None:
06:0156                         from .inference import infer_hands_from_image, load_runtime
06:0157                         # External WiLoR resolves MANO relative to cwd; CLI establishes ROOT.
06:0158                         runtime = load_runtime(args.checkpoint, args.cfg, args.detector,
06:0159                                              args.device)
06:0160                     import torch

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

06:0162         torch.set_float32_matmul_precision("highest")
06:0163         torch.backends.cudnn.allow_tf32 = False
06:0164         decoder = ManoDecoder(runtime.model.mano)
06:0165         import torch
06:0166         invocation["runtime"] = {"torch": torch.__version__, "device":
str(runtime.device),
06:0167                                     "cuda": torch.version.cuda,
"float32_matmul_precision": torch.get_float32_matmul_precision()}
06:0168         tick = time.perf_counter()
06:0169         hands = infer_hands_from_image(image, runtime,
detector_conf=args.detector_conf,
06:0170         rescale_factor=args.rescale_factor, batch_size=args.batch_size,
06:0171         include_vertices=True, include_mano=True)
06:0172         for hand in hands:
06:0173             hand["mano"] = encode_hand(hand)
06:0174             hand["roundtrip"] = decoder.check(hand["mano"], hand)
06:0175             hand["quality"] = inspect_hand(hand)
06:0176             record = {"schema": "egohand3d.frame/1", **metadata, "width":
int(image.shape[1]),
06:0177                     "height": int(image.shape[0]), "hands": hands,
06:0178                     "timing": {"inference_and_validation_s": time.perf_counter() - tick}}
06:0179             artifacts = export_frame(directory, record, image, runtime.model.mano.faces,
06:0180                                     not args.no_overlay, not args.no_mesh)
06:0181             elapsed = time.perf_counter() - tick
06:0182             store.finish(sample_id, elapsed, f"records/{sample_id}.json", artifacts)
06:0183             invocation["processed"] += 1
06:0184             print(f"[process] {sample_id}: {len(hands)} hands, {elapsed:.3f}s", flush=True)
06:0185             break
06:0186         except Exception as exc:
06:0187             store.fail(sample_id, f"{type(exc).__name__}: {exc}")
06:0188             print(f"[failed] {sample_id} attempt {attempt+1}: {exc}", flush=True)
06:0189             if attempt == args.retries and args.fail_fast:
06:0190                 raise
06:0191         else:
06:0192             counts = store.summary()["counts"]
06:0193             store.set("state", "completed_with_errors" if counts.get("failed", 0) else "completed")
06:0194     except KeyboardInterrupt:
06:0195         store.set("state", "cancelled")
06:0196         print("[cancelled] Completed artifacts are retained; use --resume.", flush=True)
06:0197     except Exception:
06:0198         store.set("state", "failed")
06:0199         raise
06:0200     finally:
06:0201         signal.signal(signal.SIGTERM, previous_sigterm)
06:0202         invocation["wall_seconds"] = time.perf_counter() - started
06:0203         store.set("updated", now())
06:0204         summary = write_run_summary(directory, store, invocation)
06:0205         from .reports import build_run_report
06:0206         build_run_report(directory)
06:0207     return summary
07:0001 """Association and explicit postprocessing of observed hand sequences.
07:0003 This is an application-level baseline, not an identity recognition model.
07:0004 Interpolated records are marked and remain separate from detector observations.
07:0005 """

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

07:0006 from __future__ import annotations
07:0008 import copy
07:0009 import math
07:0010 from pathlib import Path
07:0012 import numpy as np
07:0013 from scipy.optimize import linear_sum_assignment
07:0015 from .mano_codec import ManoDecoder, interpolate_packets
07:0016 from .storage import file_hash, load_json, save_json
07:0019 def association_cost(a: dict, b: dict) -> float:
07:0020     if a["hand_side"] != b["hand_side"]:
07:0021         return 1e6
07:0022     x, y = np.asarray(a["bbox_xyxy"]), np.asarray(b["bbox_xyxy"])
07:0023     intersection = np.maximum(0, np.minimum(x[2:], y[2:]) - np.maximum(x[:2], y[:2])).prod()
07:0024     area_x = np.maximum(0, x[2:] - x[:2]).prod()
07:0025     area_y = np.maximum(0, y[2:] - y[:2]).prod()
07:0026     iou = intersection / max(area_x + area_y - intersection, 1.)
07:0027     scale = max(np.linalg.norm(x[2:] - x[:2]), np.linalg.norm(y[2:] - y[:2]), 1.)
07:0028     distance = np.linalg.norm((x[:2] + x[2:] - y[:2] - y[2:]) / 2) / scale
07:0029     if distance > 1.5:
07:0030         return 1e6
07:0031     return float(0.6 * (1 - iou) + 0.4 * distance)
07:0034 class HandTracker:
07:0035     def __init__(self, max_gap: int = 3, smoothing_seconds: float = 0.08):
07:0036         if max_gap < 0 or smoothing_seconds < 0:
07:0037             raise ValueError("max_gap and smoothing_seconds must be nonnegative")
07:0038         self.max_gap = max_gap
07:0039         self.tau = smoothing_seconds
07:0040         self.tracks = {}
07:0041         self.next_id = 1
07:0042         self.previous_time = None
07:0044     def update(self, hands: list[dict], index: int, timestamp: float):
07:0045         if not math.isfinite(timestamp) or (self.previous_time is not None and timestamp <=
self.previous_time):
07:0046             raise ValueError("Sequence timestamps must be finite and strictly increasing")
07:0047         self.previous_time = timestamp
07:0048         self.tracks = {k: v for k, v in self.tracks.items() if index - v["index"] <= self.max_gap +
1}
07:0049         keys = list(self.tracks)
07:0050         assignments = {}
07:0051         if keys and hands:
07:0052             cost = np.array([[association_cost(self.tracks[k]["hand"], h) for h in hands] for k in
keys])
07:0053             rr, cc = linear_sum_assignment(cost)
07:0054             assignments = {int(c): keys[int(r)] for r, c in zip(rr, cc) if cost[r, c] < 1.1}
07:0055         output = []
07:0056         gaps = []
07:0057         for position, hand in enumerate(hands):
07:0058             track_id = assignments.get(position)
07:0059             packet = hand["mano"]
07:0060             previous = self.tracks.get(track_id)
07:0061             if previous is None:
07:0062                 track_id = self.next_id
07:0063                 self.next_id += 1

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

07:0064         filtered = packet
07:0065     else:
07:0066         elapsed = timestamp - previous["timestamp"]
07:0067         # A long time gap should not retain a stale pose, even with few samples.
07:0068         weight = 1 - math.exp(-elapsed / self.tau) if self.tau else 1.
07:0069         filtered = interpolate_packets(previous["packet"], packet, weight)
07:0070         if index - previous["index"] > 1:
07:0071             gaps.append({"track_id": track_id, "start": previous["index"], "end": index,
07:0072                         "start_time": previous["timestamp"], "end_time": timestamp,
07:0073                         "first": previous["packet"], "second": filtered})
07:0074         current = {**copy.deepcopy(hand), "track_id": track_id, "observation": "detected",
07:0075                  "processed_mano": filtered}
07:0076         self.tracks[track_id] = {"hand": hand, "packet": filtered, "index": index, "timestamp":
timestamp}
07:0077         output.append(current)
07:0078         return output, gaps
07:0081 def geometry_record(decoder: ManoDecoder, packet: dict) -> dict:
07:0082     decoded = decoder.decode(packet)
07:0083     joints = decoded["joints_camera"]
07:0084     return {"joints_3d_root": decoded["joints_model"].tolist(), "joints_3d_camera":
joints.tolist(),
07:0085           "joints_2d_xy": decoded["joints_2d"].tolist(), "cam_t_full":
packet["translation_camera"],
07:0086           "positive_depth": decoded["positive_depth"].tolist()}
07:0089 def process_sequence(run: Path, out: Path, mano_dir: Path, max_gap: int = 3,
07:0090                   smoothing_seconds: float = 0.08, fill_gaps: bool = False) -> dict:
07:0091     if out.exists() and any(out.iterdir()):
07:0092         raise ValueError("Use a new, empty sequence output directory")
07:0093     run_index = load_json(run / "index.json")["records"]
07:0094     if not run_index:
07:0095         raise ValueError("No completed frames")
07:0096     groups = {}
07:0097     for entry in run_index:
07:0098         if entry["timestamp_s"] is None:
07:0099             raise ValueError("Timestamps required: process a video or use --sequence-fps for image
sequences")
07:0100         groups.setdefault(entry["sequence_id"], []).append(entry)
07:0101     for entries in groups.values():
07:0102         entries.sort(key=lambda r: r["frame_index"])
07:0103     decoder = ManoDecoder.from_directory(mano_dir)
07:0104     out.mkdir(parents=True, exist_ok=True)
07:0105     count_observed = count_filled = 0
07:0106     new_index = []
07:0107     track_count = 0
07:0108     source_hashes = {}
07:0109     traces = []
07:0110     source_spacing = {}
07:0111     for sequence_id, entries in groups.items():
07:0112         tracker = HandTracker(max_gap, smoothing_seconds)
07:0113         source_spacing[sequence_id] = float(np.median(np.diff([e["timestamp_s"] for e in
entries]))) if len(entries) > 1 else None
07:0114         for index, entry in enumerate(entries):
07:0115             source_hashes[entry["record"]] = file_hash(run / entry["record"])
07:0116             record = load_json(run / entry["record"])
07:0117             hands, gaps = tracker.update(record["hands"], index, entry["timestamp_s"])

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

07:0118         for hand in hands:
07:0119             hand["processed"] = geometry_record(decoder, hand["processed_mano"])
07:0120             traces.append({"sequence_id": sequence_id, "track_id": hand["track_id"],
07:0121                             "frame_index": entry["frame_index"], "timestamp_s":
entry["timestamp_s"],
07:0122                             "raw_joints": hand["joints_3d_camera"],
07:0123                             "processed_joints": hand["processed"]["joints_3d_camera"]})
07:0124         record["hands"] = hands
07:0125         record["postprocessing"] = {"smoothing_seconds": smoothing_seconds,
07:0126                                     "raw_predictions_preserved": True, "fill_gaps": fill_gaps}
07:0127         output_path = out / entry["record"]
07:0128         save_json(output_path, record)
07:0129         new_index.append(dict(entry))
07:0130         count_observed += len(hands)
07:0131         if fill_gaps:
07:0132             for gap in gaps:
07:0133                 for missing_index in range(gap["start"] + 1, gap["end"]):
07:0134                     missing_entry = entries[missing_index]
07:0135                     path = out / missing_entry["record"]
07:0136                     missing_frame = load_json(path)
07:0137                     weight = (missing_entry["timestamp_s"] - gap["start_time"]) /
(gap["end_time"] - gap["start_time"])
07:0138                     packet = interpolate_packets(gap["first"], gap["second"], weight)
07:0139                     missing_frame["hands"].append({"track_id": gap["track_id"],
07:0140                                                     "hand_side": packet["hand_side"], "observation": "interpolated",
07:0141                                                     "processed_mano": packet, "processed": geometry_record(decoder,
packet),
07:0142                                                     "interpolation": {"from_sample": entries[gap["start"]]["sample_id"],
07:0143                                                         "to_sample": entries[gap["end"]]["sample_id"],
"weight": weight}})
07:0144                     save_json(path, missing_frame)
07:0145                     count_filled += 1
07:0146         track_count += tracker.next_id - 1
07:0147         # Compute velocities only on consecutive detected observations, never gap fills.
07:0148         speeds = {"raw": [], "processed": []}
07:0149         previous = {}
07:0150         for row in traces:
07:0151             key = (row["sequence_id"], row["track_id"])
07:0152             prior = previous.get(key)
07:0153             dt = row["timestamp_s"] - prior["timestamp_s"] if prior else None
07:0154             spacing = source_spacing[row["sequence_id"]]
07:0155             if prior and spacing is not None and dt <= 1.5 * spacing:
07:0156                 for variant in speeds:
07:0157                     delta = np.asarray(row[f"{variant}_joints"]) -
np.asarray(prior[f"{variant}_joints"])
07:0158                     speeds[variant].append(float(np.linalg.norm(delta, axis=1).mean() / dt))
07:0159             previous[key] = row
07:0160         summary = {"schema": "egohand3d.sequence/1", "source_run": str(run.resolve()),
07:0161                   "frames": len(new_index), "sequences": len(groups), "tracks": track_count,
07:0162                   "detected_hands": count_observed, "interpolated_hands": count_filled,
07:0163                   "max_gap_samples": max_gap, "smoothing_seconds": smoothing_seconds,
07:0164                   "mean_joint_speed_m_s": {k: float(np.mean(v)) if v else None for k, v in
speeds.items()}},
07:0165         "speed_note": "Descriptive motion statistic, not accuracy or a ground-truth jitter
metric",
07:0166         "source_record_sha256": source_hashes}
07:0167         save_json(out / "index.json", {"schema": "egohand3d.index/1", "records": new_index})

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

07:0168     save_json(out / "summary.json", summary)
07:0169     from .reports import build_sequence_report
07:0170     build_sequence_report(out, traces, summary)
07:0171     return summary
08:0001     """Convert existing, trusted HOI4D annotations into an explicit evaluation manifest."""
08:0002     from __future__ import annotations
08:0004     from pathlib import Path
08:0005     import pickle
08:0007     import numpy as np
08:0009     from .storage import file_hash, load_json, save_json
08:0012     def prepare_hoi4d(run: Path, sample_dir: Path, out: Path) -> dict:
08:0013         manifest = load_json(run / "manifest.json")
08:0014         config = load_json(run / "run_config.json")
08:0015         source_manifest = sample_dir / "meta/manifest.json"
08:0016         annotations = load_json(source_manifest)
08:0017         by_path = {str((sample_dir / a["image"]).resolve()): a for a in annotations}
08:0018         if len(by_path) != len(annotations):
08:0019             raise ValueError("Duplicate HOI4D image entries")
08:0020         media = manifest["files"]
08:0021         if any(item["kind"] != "image" for item in media):
08:0022             raise ValueError("This adapter expects an image run")
08:0023         limit = config["options"]["limit"]
08:0024         if limit:
08:0025             media = media[:limit]
08:0026         frames = []
08:0027         labels = {}
08:0028         for item in media:
08:0029             key = item["identity"]["path"]
08:0030             if key not in by_path:
08:0031                 raise ValueError(f"Input lacks a GT mapping: {key}")
08:0032             entry = by_path[key]
08:0033             label_path = sample_dir / entry["label"]
08:0034             # HOI4D stores annotations as pickle. Only use the user's trusted local dataset.
08:0035             with label_path.open("rb") as stream:
08:0036                 annotation = pickle.load(stream)
08:0037             points = np.asarray(annotation["kps2D"], dtype=float)
08:0038             if points.shape != (21, 2) or not np.isfinite(points).all():
08:0039                 raise ValueError(f"Invalid 2D label: {label_path}")
08:0040             minimum, maximum = points.min(0), points.max(0)
08:0041             padding = np.maximum(maximum - minimum, 1.) * 0.1
08:0042             bbox = np.concatenate([minimum - padding, maximum + padding])
08:0043             frames.append({"sample_id": item["id"], "source_sample_id": entry["sample_id"],
08:0044                           "hands": [{"hand_side": entry["hand_side"], "bbox_xyxy": bbox.tolist(),
08:0045                                     "bbox_source": "GT 2D joint extent with 10% padding per side",
08:0046                                     "joints_2d_xy": points.tolist(), "valid": [True] * 21}])
08:0047             labels[str(label_path.resolve())] = file_hash(label_path)
08:0048             result = {"schema": "egohand3d.ground_truth/1", "joint_order": "OpenPose21", "length_unit":
08:0049 "m",
08:0049                 "camera_axes": "x_right_y_down_z_forward", "labels_complete": False,
08:0050                 "label_scope": "One annotated target hand per image; other visible hands may be
unlabelled",
08:0051                 "source_manifest_sha256": file_hash(source_manifest), "label_sha256": labels,

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

08:0052         "note": "2D evaluation only; MANO GT requires a separately verified joint/coordinate
conversion",
08:0053         "frames": frames}
08:0054     save_json(out, result)
08:0055     return {"frames": len(frames), "out": str(out), "ground_truth_sha256": file_hash(out)}
09:0001     """Explicit evaluation with fixed manifests and prediction-independent matching."""
09:0002     from __future__ import annotations
09:0004     import numpy as np
09:0005     from scipy.optimize import linear_sum_assignment
09:0008     def similarity_align(prediction, target) -> np.ndarray:
09:0009         x, y = np.asarray(prediction, dtype=float), np.asarray(target, dtype=float)
09:0010         if x.shape != y.shape or x.ndim != 2 or x.shape[1] != 3 or len(x) < 3:
09:0011             raise ValueError("Alignment requires matching Nx3 arrays, N >= 3")
09:0012         xc, yc = x - x.mean(0), y - y.mean(0)
09:0013         variance = np.sum(xc * xc)
09:0014         if variance < 1e-12 or np.linalg.matrix_rank(xc) < 2 or np.linalg.matrix_rank(yc) < 2:
09:0015             raise ValueError("Degenerate joints cannot define a similarity alignment")
09:0016         u, singular, vt = np.linalg.svd(xc.T @ yc)
09:0017         correction = np.eye(3)
09:0018         correction[-1, -1] = np.sign(np.linalg.det(u @ vt))
09:0019         rotation = u @ correction @ vt
09:0020         scale = np.sum(singular * np.diag(correction)) / variance
09:0021         return scale * xc @ rotation + y.mean(0)
09:0024     def bbox_iou(first, second) -> float:
09:0025         a, b = np.asarray(first, dtype=float), np.asarray(second, dtype=float)
09:0026         if a.shape != (4,) or b.shape != (4,) or not np.isfinite([a, b]).all():
09:0027             raise ValueError("Bounding boxes must be finite xyxy arrays")
09:0028         intersection = np.maximum(0, np.minimum(a[2:], b[2:]) - np.maximum(a[:2], b[:2])).prod()
09:0029         union = np.maximum(0, a[2:] - a[:2]).prod() + np.maximum(0, b[2:] - b[:2]).prod() -
intersection
09:0030         return float(intersection / union) if union > 0 else 0.
09:0033     def match_hands(predictions: list[dict], targets: list[dict], threshold: float = 0.1):
09:0034         """One-to-one same-side IoU matching; never choose by joint error."""
09:0035         observed = [i for i, p in enumerate(predictions) if p.get("observation", "detected") ==
"detected"]
09:0036         if not observed or not targets:
09:0037             return []
09:0038         cost = np.full((len(observed), len(targets)), 1e6)
09:0039         for i, pi in enumerate(observed):
09:0040             for j, target in enumerate(targets):
09:0041                 prediction = predictions[pi]
09:0042                 if prediction["hand_side"] == target["hand_side"]:
09:0043                     overlap = bbox_iou(prediction["bbox_xyxy"], target["bbox_xyxy"])
09:0044                     if overlap >= threshold:
09:0045                         cost[i, j] = 1 - overlap
09:0046         ii, jj = linear_sum_assignment(cost)
09:0047         return [(observed[int(i)], int(j)) for i, j in zip(ii, jj) if cost[i, j] < 1e6]
09:0050     def error_summary(values) -> dict:
09:0051         array = np.asarray(values, dtype=float)
09:0052         if not array.size:
09:0053             return {"count": 0, "mean": None, "median": None, "p95": None}
09:0054         return {"count": int(array.size), "mean": float(array.mean()), "median":
float(np.median(array)),
09:0055                 "p95": float(np.percentile(array, 95))}

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

09:0058 def evaluate_frames(targets: list[dict], predictions: dict, variant: str = "raw",
09:0059                      match_iou: float = 0.1, pck_thresholds=(5., 10., 20., 50.)) -> dict:
09:0060     if variant not in {"raw", "processed"} or not 0 <= match_iou <= 1:
09:0061         raise ValueError("Invalid variant/matching threshold")
09:0062     if not pck_thresholds or any(t <= 0 for t in pck_thresholds):
09:0063         raise ValueError("PCK thresholds must be positive")
09:0064     ids = [str(frame["sample_id"]) for frame in targets]
09:0065     if len(set(ids)) != len(ids):
09:0066         raise ValueError("Duplicate ground-truth sample IDs")
09:0067     errors = {"2d_px": [], "camera_mpjpe_mm": [], "root_mpjpe_mm": [], "pa_mpjpe_mm": []}
09:0068     gt_hands = matched = detected = total_2d = degenerate_pa = missing_frames = 0
09:0069     details = []
09:0070     for frame in targets:
09:0071         record = predictions.get(str(frame["sample_id"]))
09:0072         hands = record.get("hands", []) if record else []
09:0073         missing_frames += record is None
09:0074         labels = frame["hands"]
09:0075         gt_hands += len(labels)
09:0076         detected += sum(h.get("observation", "detected") == "detected" for h in hands)
09:0077         valid_labels = []
09:0078         for target in labels:
09:0079             if target["hand_side"] not in {"left", "right"}:
09:0080                 raise ValueError("GT hand_side must be left/right")
09:0081             bbox_iou(target["bbox_xyxy"], target["bbox_xyxy"])
09:0082             valid = np.asarray(target.get("valid", [True] * 21), dtype=bool)
09:0083             if valid.shape != (21,):
09:0084                 raise ValueError("GT valid mask must have 21 entries")
09:0085             for key, dim in [("joints_2d_xy", 2), ("joints_3d_camera", 3)]:
09:0086                 if key in target:
09:0087                     points = np.asarray(target[key], dtype=float)
09:0088                     if points.shape != (21, dim) or not np.isfinite(points[valid]).all():
09:0089                         raise ValueError(f"Invalid GT {key}")
09:0090             if "joints_2d_xy" in target:
09:0091                 total_2d += int(valid.sum())
09:0092             valid_labels.append(valid)
09:0093     pairs = match_hands(hands, labels, match_iou)
09:0094     matched += len(pairs)
09:0095     for pi, gi in pairs:
09:0096         target = labels[gi]
09:0097         hand = hands[pi]
09:0098         predicted = hand if variant == "raw" else hand.get("processed")
09:0099         if predicted is None:
09:0100             raise ValueError("Processed evaluation requires processed geometry for every
matched hand")
09:0101         valid = valid_labels[gi]
09:0102         row = {"sample_id": frame["sample_id"], "gt_hand_index": gi, "prediction_index": pi}
09:0103         if "joints_2d_xy" in target:
09:0104             points = np.asarray(predicted["joints_2d_xy"], dtype=float)
09:0105             if points.shape != (21, 2) or not np.isfinite(points).all():
09:0106                 raise ValueError("Invalid predicted 2D joints")
09:0107             err = np.linalg.norm(points[valid] - np.asarray(target["joints_2d_xy"])[valid],
axis=1)

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

09:0108         errors["2d_px"].extend(err.tolist())
09:0109         row["mean_2d_px"] = float(err.mean()) if len(err) else None
09:0110     if "joints_3d_camera" in target:
09:0111         points = np.asarray(predicted["joints_3d_camera"], dtype=float)
09:0112         truth = np.asarray(target["joints_3d_camera"], dtype=float)
09:0113         if points.shape != (21, 3) or not np.isfinite(points).all():
09:0114             raise ValueError("Invalid predicted 3D joints")
09:0115         camera = np.linalg.norm(points[valid] - truth[valid], axis=1) * 1000
09:0116         errors["camera_mpjpe_mm"].extend(camera.tolist())
09:0117         if valid[0]:
09:0118             root = np.linalg.norm((points - points[0])[valid] - (truth - truth[0])[valid],
axis=1) * 1000
09:0119             errors["root_mpjpe_mm"].extend(root.tolist())
09:0120         try:
09:0121             aligned = similarity_align(points[valid], truth[valid])
09:0122             pa = np.linalg.norm(aligned - truth[valid], axis=1) * 1000
09:0123             errors["pa_mpjpe_mm"].extend(pa.tolist())
09:0124         except ValueError:
09:0125             degenerate_pa += 1
09:0126         details.append(row)
09:0127         pixel_errors = np.asarray(errors["2d_px"])
09:0128         pck = {str(float(threshold)): {"matched_only": float(np.mean(pixel_errors <= threshold)) if
pixel_errors.size else None,
09:0129         "all_gt_joints": int(np.sum(pixel_errors <= threshold)) / total_2d if total_2d else
None}
09:0130         for threshold in sorted(set(pck_thresholds))}
09:0131         return {"schema": "egohand3d.benchmark/1", "variant": variant, "frames": len(targets),
09:0132         "missing_prediction_frames": missing_frames, "gt_hands": gt_hands, "predicted_hands":
detected,
09:0133         "matched_hands": matched, "missed_hands": gt_hands - matched, "unmatched_predictions":
detected - matched,
09:0134         "hand_recall": matched / gt_hands if gt_hands else None,
09:0135         "hand_precision": matched / detected if detected else None,
09:0136         "metrics": {key: error_summary(value) for key, value in errors.items()},
09:0137         "pck_px": pck, "num_gt_2d_joints": total_2d, "degenerate_pa_hands": degenerate_pa,
09:0138         "protocol": {"matching": "one_to_one_same_side_bbox_iou", "minimum_iou": match_iou,
09:0139         "interpolated_hands": "excluded", "3d_unit": "m", "joint_order":
"OpenPose21",
09:0140         "mean_error_denominator": "valid joints in matched hands; report recall
alongside",
09:0141         "all_gt_pck": "unmatched/missing hands count as incorrect"}, "details":
details}
10:0001 """Static reports, plots, and portable result archives; no web application."""
10:0002 from __future__ import annotations
10:0003 import csv
10:0004 import html
10:0005 import io
10:0006 import json
10:0007 from pathlib import Path
10:0008 import zipfile
10:0009 import numpy as np
10:0010 from .storage import atomic_text, file_hash, load_json, save_json
10:0011 def write_report(directory, title, paragraphs, rows, columns):
10:0012     markdown = [f"# {title}", "", *paragraphs, "", "|" + "|".join(columns) + "|",
10:0013     "|" + "|".join(["---"] * len(columns)) + "|"]
10:0014     markdown += ["|" + "|".join(str(v) for v in row) + "|" for row in rows]
10:0015     atomic_text(directory / "report.md", "\n".join(markdown) + "\n")
10:0016     table = "<tr>" + "".join(f"<th>{html.escape(c)}</th>" for c in columns) + "</tr>"

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

10:0022     table += ".join("<tr>" + ".join(f"<td>{html.escape(str(v))}</td>" for v in row) + "</tr>" for
row in rows)
10:0023     images = ".join(f'<img src='{html.escape(p.name)}' alt='{html.escape(p.stem)}'>'
10:0024         for p in sorted(directory.glob("chart_*.png"))
10:0025     document = f'<!doctype html><html lang="en"><meta
charset="utf-8"><title>{html.escape(title)}</title>
10:0026 <style>body{{max-width:1050px;margin:40px auto;padding:0 20px;font:16px/1.6 sans-serif;color:#223}}
10:0027 table{{border-collapse:collapse}}td,th{{border:1px solid
#ccd;padding:8px;text-align:left}}img{{max-width:100%}}</style>
10:0028 <h1>{html.escape(title)}</h1>{''.join('<p>'+html.escape(p)+'</p>' for p in
paragraphs)}<table>{table}</table>{images}</html>'"""
10:0029     atomic_text(directory / "report.html", document)
10:0032 def pyplot():
10:0033     import matplotlib
10:0034     matplotlib.use("Agg")
10:0035     import matplotlib.pyplot as plt
10:0036     return plt
10:0039 def build_run_report(directory: Path):
10:0040     summary = load_json(directory / "summary.json")
10:0041     flags = summary["quality_flags"]
10:0042     if flags:
10:0043         plt = pyplot()
10:0044         fig, ax = plt.subplots(figsize=(9, 4), layout="constrained")
10:0045         ax.barh(list(flags), list(flags.values()), color="#437a9e")
10:0046         ax.set(xlabel="Hand observations flagged", title="Quality diagnostics (not accuracy)")
10:0047         fig.savefig(directory / "chart_quality.png", dpi=160)
10:0048         plt.close(fig)
10:0049     rows = [("Run state", summary["state"]), ("Processed frames", summary["frames_completed"]),
10:0050         ("Frames containing predictions", summary["frames_with_hands"]), ("Hands",
summary["hands"]),
10:0051         ("MANO consistency failures", summary["roundtrip_failures"]),
10:0052         ("Failed tasks", len(summary["failures"])),
10:0053         ("Mean frame processing seconds", summary["mean_processing_seconds"])]
10:0054     write_report(directory, "EgoHand3D processing report", [
10:0055         "Generated from actual processing records. Detection coverage is not recall without ground
truth.",
10:0056         "MANO consistency checks serialization/reconstruction, not pose estimation accuracy.",
10:0057         "run_config.json records asset/input identities, code hashes and options. index.csv maps
artifacts to inputs.",
10:0058         "Images and meshes are stored in overlays/ and meshes/."], rows, ["Item", "Value"])
10:0061 def build_sequence_report(directory: Path, traces: list[dict], summary: dict):
10:0062     if traces:
10:0063         plt = pyplot()
10:0064         fig, axes = plt.subplots(2, 1, figsize=(10, 6), layout="constrained")
10:0065         chosen = (traces[0]["sequence_id"], traces[0]["track_id"])
10:0066         subset = [t for t in traces if (t["sequence_id"], t["track_id"]) == chosen]
10:0067         for variant in ["raw", "processed"]:
10:0068             wrist = np.array([t[f"{variant}_joints"][0] for t in subset])
10:0069             times = np.array([t["timestamp_s"] for t in subset])
10:0070             axes[0].plot(times, wrist[:, 0], label=variant)
10:0071             axes[1].plot(times, wrist[:, 2], label=variant)
10:0072         axes[0].set_ylabel("Wrist x (m)")
10:0073         axes[1].set_ylabel("Wrist z (m)")
10:0074         axes[1].set_xlabel("Time (s)")
10:0075         for ax in axes:
10:0076             ax.legend()
10:0077         axes[0].set_title(f"Sequence {chosen[0]}, track {chosen[1]} – observed frames")

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

10:0078         fig.savefig(directory / "chart_trajectory.png", dpi=160)
10:0079         plt.close(fig)
10:0080         write_report(directory, "EgoHand3D sequence report", [
10:0081             "Raw predictions are retained beside processed MANO and geometry.",
10:0082             "Interpolated hands are explicitly labelled and excluded from detection and accuracy
evaluation.",
10:0083             "Lower motion speed does not imply better accuracy; evaluate against ground truth.",
10:0084             [(key, summary[key]) for key in ["frames", "sequences", "tracks", "detected_hands",
"interpolated_hands",
10:0085                                     "smoothing_seconds", "mean_joint_speed_m_s"]], ["Item",
"Value"])
10:0088 def build_benchmark_report(directory: Path, result: dict):
10:0089     save_json(directory / "summary.json", result)
10:0090     buffer = io.StringIO()
10:0091     writer = csv.DictWriter(buffer, fieldnames=["sample_id", "gt_hand_index", "prediction_index",
"mean_2d_px"])
10:0092     writer.writeheader()
10:0093     writer.writerows(result["details"])
10:0094     atomic_text(directory / "details.csv", buffer.getvalue())
10:0095     plt = pyplot()
10:0096     fig, ax = plt.subplots(figsize=(7, 4), layout="constrained")
10:0097     for denominator in ["matched_only", "all_gt_joints"]:
10:0098         pairs = [(float(k), result["pck_px"][k][denominator]) for k in sorted(result["pck_px"],
key=float)
10:0099                 if result["pck_px"][k][denominator] is not None]
10:0100         if pairs:
10:0101             ax.plot(*zip(*pairs), marker="o", label=denominator)
10:0102         ax.set(xlabel="Threshold (px)", ylabel="PCK", ylim=(0, 1.02), title="2D PCK with explicit
denominators")
10:0103         if ax.lines:
10:0104             ax.legend()
10:0105         fig.savefig(directory / "chart_pck.png", dpi=160)
10:0106         plt.close(fig)
10:0107         rows = [(key, result[key]) for key in ["frames", "gt_hands", "matched_hands", "missed_hands",
"unmatched_predictions", "hand_recall", "hand_precision"]]
10:0108         rows += [(key, value["mean"]) for key, value in result["metrics"].items()]
10:0109         write_report(directory, "EgoHand3D benchmark report", [
10:0110             "Matching uses same-side bounding-box IoU, not the smallest ground-truth joint error.",
10:0111             "Joint errors use matched hands. All-GT PCK counts missing hands as failures.",
10:0112             f"Ground-truth manifest SHA-256: {result['ground_truth_sha256']}", rows, ["Metric",
"Value"])
10:0116 def compare_benchmarks(paths: list[Path], out: Path):
10:0117     results = [load_json(p) for p in paths]
10:0118     if len(results) < 2:
10:0119         raise ValueError("At least two benchmark summaries are required")
10:0120     reference = results[0]
10:0121     for result in results:
10:0122         if result.get("schema") != "egohand3d.benchmark/1":
10:0123             raise ValueError("Comparison requires benchmark summaries")
10:0124         for key in ["ground_truth_sha256", "protocol", "evaluator_sha256"]:
10:0125             if result[key] != reference[key]:
10:0126                 raise ValueError(f"Cannot compare different {key}")
10:0127             if list(result["pck_px"]) != list(reference["pck_px"]):
10:0128                 raise ValueError("PCK thresholds differ")
10:0129     if out.exists() and any(out.iterdir()):
10:0130         raise ValueError("Comparison output directory must be empty")
10:0131     out.mkdir(parents=True, exist_ok=True)

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

10:0132     rows = [(str(path), result["variant"], result["hand_recall"],
10:0133               result["metrics"]["2d_px"]["mean"],
10:0134               result["metrics"]["root_mpjpe_mm"]["mean"], result["metrics"]["pa_mpjpe_mm"]["mean"])
10:0135               for path, result in zip(paths, results)]
10:0136     columns = ["Run", "Variant", "Recall", "2D px", "Root MPJPE mm", "PA-MPJPE mm"]
10:0137     write_report(out, "EgoHand3D benchmark comparison", ["Identical GT manifest, matching rules and
10:0138               evaluator verified."], rows, columns)
10:0139     save_json(out / "comparison.json", {"inputs": [str(p) for p in paths], "columns": columns,
10:0140               "rows": rows,
10:0141               "ground_truth_sha256": reference["ground_truth_sha256"]})
10:0142     return {"runs": len(rows), "report": str(out / "report.html")}
10:0143 def package_results(directory: Path, destination: Path) -> dict:
10:0144     if not (directory / "summary.json").is_file():
10:0145         raise ValueError("Result directory needs summary.json")
10:0146     if destination.exists():
10:0147         raise FileExistsError(f"Archive already exists: {destination}")
10:0148     allowed = {".json", ".jsonl", ".csv", ".md", ".html", ".png", ".jpg", ".obj", ".mp4"}
10:0149     files = []
10:0150     for path in sorted(directory.rglob("*")):
10:0151         if path.is_symlink():
10:0152             raise ValueError(f"Refusing to package symlink: {path}")
10:0153         if path.is_file() and path.suffix in allowed:
10:0154             files.append(path)
10:0155     manifest = {str(p.relative_to(directory)): {"bytes": p.stat().st_size, "sha256": file_hash(p)}
10:0156               for p in files}
10:0157     destination.parent.mkdir(parents=True, exist_ok=True)
10:0158     temporary = destination.with_suffix(destination.suffix + ".partial")
10:0159     try:
10:0160         with zipfile.ZipFile(temporary, "w", compression=zipfile.ZIP_DEFLATED) as archive:
10:0161             for path in files:
10:0162                 archive.write(path, str(path.relative_to(directory)))
10:0163                 archive.writestr("ARTIFACT_MANIFEST.json", json.dumps(manifest, indent=2))
10:0164             temporary.replace(destination)
10:0165     finally:
10:0166         temporary.unlink(missing_ok=True)
10:0167     return {"archive": str(destination), "files": len(files), "sha256": file_hash(destination)}
11:0001 """Offline side-by-side videos comparing raw observations and processed poses."""
11:0002 from __future__ import annotations
11:0003 from pathlib import Path
11:0004 import cv2
11:0005 import numpy as np
11:0006 from .storage import load_json
11:0007 from .visualization import draw_hand_skeleton, draw_label
11:0008 def export_sequence_videos(directory: Path) -> list[str]:
11:0009     entries = load_json(directory / "index.json")["records"]
11:0010     groups = {}
11:0011     for entry in entries:
11:0012         groups.setdefault(entry["sequence_id"], []).append(entry)
11:0013     outputs = []
11:0014     for sequence_id, group in groups.items():
11:0015         group.sort(key=lambda e: e["timestamp_s"])
11:0016         times = np.array([e["timestamp_s"] for e in group])
11:0017         differences = np.diff(times)
11:0018         if not len(differences):

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

11:0023         continue
11:0024         if not np.allclose(differences, differences[0], rtol=1e-3, atol=1e-6):
11:0025             raise ValueError("Video export requires evenly spaced frames; JSON and plots support
irregular timestamps")
11:0026         fps = float(1 / differences[0])
11:0027         output = directory / f"{sequence_id}_comparison.mp4"
11:0028         writer = None
11:0029         capture = None
11:0030         capture_path = None
11:0031         try:
11:0032             for entry in group:
11:0033                 record = load_json(directory / entry["record"])
11:0034                 source = Path(record["source_path"])
11:0035                 if record["time_source"] == "container_fps":
11:0036                     if str(source) != capture_path:
11:0037                         if capture is not None:
11:0038                             capture.release()
11:0039                             capture = cv2.VideoCapture(str(source))
11:0040                             capture_path = str(source)
11:0041                             capture.set(cv2.CAP_PROP_POS_FRAMES, record["frame_index"])
11:0042                             ok, image = capture.read()
11:0043                             if not ok:
11:0044                                 raise ValueError(f"Cannot reread source video frame
{record['frame_index']}")
11:0045                             else:
11:0046                                 image = cv2.imread(str(source))
11:0047                                 if image is None:
11:0048                                     raise ValueError(f"Cannot reread source image {source}")
11:0049                             panels = [image.copy(), image.copy()]
11:0050                             for hand in record["hands"]:
11:0051                                 observed = hand.get("observation") == "detected"
11:0052                                 if observed:
11:0053                                     draw_hand_skeleton(panels[0], np.asarray(hand["joints_2d_xy"]))
11:0054                                     points = np.asarray(hand["processed"]["joints_2d_xy"])
11:0055                                     draw_hand_skeleton(panels[1], points)
11:0056                                     location = tuple(np rint(points[0]).astype(int))
11:0057                                     draw_label(panels[1], f"ID {hand['track_id']} {'detected' if observed else
'INTERPOLATED'}",
11:0058                                             location, (20, 230, 100) if observed else (20, 120, 250))
11:0059                             for panel, title in zip(panels, ["Raw model observations", "Processed MANO; gaps
explicitly labelled"]):
11:0060                                 draw_label(panel, title, (15, 30), (240, 240, 240))
11:0061                             combined = np.hstack(panels)
11:0062                             if writer is None:
11:0063                                 size = (combined.shape[1], combined.shape[0])
11:0064                                 writer = cv2.VideoWriter(str(output), cv2.VideoWriter_fourcc(*"mp4v"), fps,
size)
11:0065                                 if not writer.isOpened():
11:0066                                     raise OSError("Cannot initialize MP4 writer")
11:0067                                 if (combined.shape[1], combined.shape[0]) != size:
11:0068                                     raise ValueError("Sequence image dimensions changed")
11:0069                                 writer.write(combined)
11:0070                             finally:
11:0071                                 if writer is not None:
11:0072                                     writer.release()

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

11:0073         if capture is not None:
11:0074             capture.release()
11:0075             outputs.append(str(output))
11:0076         return outputs
12:0001 """Register the headless workflow in the existing command line."""
12:0002 from __future__ import annotations
12:0004 import argparse
12:0005 import json
12:0006 import math
12:0007 import os
12:0008 from pathlib import Path
12:0010 ROOT = Path(__file__).resolve().parents[1]
12:0013 def positive_int(value):
12:0014     value = int(value)
12:0015     if value < 1:
12:0016         raise argparse.ArgumentTypeError("must be positive")
12:0017     return value
12:0020 def nonnegative_int(value):
12:0021     value = int(value)
12:0022     if value < 0:
12:0023         raise argparse.ArgumentTypeError("must be nonnegative")
12:0024     return value
12:0027 def positive_float(value):
12:0028     value = float(value)
12:0029     if not math.isfinite(value) or value <= 0:
12:0030         raise argparse.ArgumentTypeError("must be finite and positive")
12:0031     return value
12:0034 def probability(value):
12:0035     value = float(value)
12:0036     if not 0 <= value <= 1:
12:0037         raise argparse.ArgumentTypeError("must be in [0,1]")
12:0038     return value
12:0041 def _execute(args):
12:0042     try:
12:0043         result = execute(args)
12:0044     except (ValueError, OSError, KeyError, RuntimeError) as exc:
12:0045         print(f"[error] {exc}")
12:0046         return 1
12:0047     print(json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False))
12:0048     if result.get("state") == "cancelled":
12:0049         return 130
12:0050     if result.get("state") in {"completed_with_errors", "failed"} or result.get("passed") is False
or result.get("roundtrip_failures", 0):
12:0051         return 1
12:0052     if args.command == "scan" and result["summary"]["unreadable"]:
12:0053         return 1
12:0054     return 0
12:0057 def execute(args):
12:0058     from .storage import file_hash, load_json, save_json
12:0059     if args.command == "scan":
12:0060         from .media import scan_media

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

12:0061         result = scan_media(args.input, args.recursive, args.hash_inputs)
12:0062         save_json(args.out, result)
12:0063         return result
12:0064     if args.command == "process":
12:0065         from .pipeline import process
12:0066         for key in ["input", "out", "checkpoint", "cfg", "detector"]:
12:0067             setattr(args, key, str(Path(getattr(args, key)).resolve()))
12:0068         if args.recursive and Path(args.out).is_relative_to(Path(args.input)):
12:0069             raise ValueError("Output must be outside a recursively scanned input directory")
12:0070         previous = Path.cwd()
12:0071         try:
12:0072             os.chdir(ROOT)
12:0073             return process(args)
12:0074         finally:
12:0075             os.chdir(previous)
12:0076     if args.command == "jobs":
12:0077         from .jobs import inspect_jobs
12:0078         return inspect_jobs(args.run, args.cancel)
12:0079     if args.command == "mano-check":
12:0080         from .mano_codec import ManoDecoder, validate_packet
12:0081         payload = load_json(args.input)
12:0082         if "hands" in payload:
12:0083             decoder = ManoDecoder.from_directory(args.mano_dir, args.device)
12:0084             checks = [decoder.check(hand["mano"], hand) for hand in payload["hands"]]
12:0085             result = {"hands": len(checks), "passed": bool(checks) and all(c["passed"] for c in
checks), "checks": checks}
12:0086         else:
12:0087             validate_packet(payload)
12:0088             result = {"passed": True, "validation": "Schema and rotation equivalence; no reference
geometry supplied"}
12:0089             save_json(args.out, result)
12:0090             return result
12:0091     if args.command == "sequence":
12:0092         from .sequence import process_sequence
12:0093         return process_sequence(args.run, args.out, args.mano_dir, args.max_gap,
args.smoothing_seconds, args.fill_gaps)
12:0094     if args.command == "prepare-gt":
12:0095         from .dataset_adapters import prepare_hoi4d
12:0096         return prepare_hoi4d(args.run, args.sample_dir, args.out)
12:0097     if args.command == "render-sequence":
12:0098         from .video_export import export_sequence_videos
12:0099         return {"videos": export_sequence_videos(args.run)}
12:0100     if args.command == "benchmark":
12:0101         from .metrics import evaluate_frames
12:0102         from .reports import build_benchmark_report
12:0103         truth = load_json(args.ground_truth)
12:0104         if truth.get("schema") != "egohand3d.ground_truth/1" or truth.get("joint_order") !=
"OpenPose21":
12:0105             raise ValueError("GT needs schema egohand3d.ground_truth/1 and joint_order OpenPose21")
12:0106         if truth.get("length_unit") != "m" or truth.get("camera_axes") !=
"x_right_y_down_z_forward":
12:0107             raise ValueError("GT must declare meters and camera axes x_right_y_down_z_forward")
12:0108         entries = load_json(args.run / "index.json")["records"]
12:0109         predictions, identities = {}, {}
12:0110         wanted = {str(f["sample_id"]) for f in truth["frames"]}

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

12:0111         for entry in entries:
12:0112             key = str(entry["sample_id"])
12:0113             if key in predictions:
12:0114                 raise ValueError("Duplicate prediction sample IDs")
12:0115             if key in wanted:
12:0116                 predictions[key] = load_json(args.run / entry["record"])
12:0117                 identities[entry["record"]] = file_hash(args.run / entry["record"])
12:0118             result = evaluate_frames(truth["frames"], predictions, args.variant, args.match_iou,
args.pck_thresholds)
12:0119             result["protocol"]["labels_complete"] = truth.get("labels_complete", True)
12:0120             result["protocol"]["label_scope"] = truth.get("label_scope", "All relevant hands
annotated")
12:0121             result["matched_prediction_fraction"] = result["hand_precision"]
12:0122             if not truth.get("labels_complete", True):
12:0123                 result["hand_precision"] = None
12:0124             result.update(ground_truth_sha256=file_hash(args.ground_truth),
12:0125                         evaluator_sha256=file_hash(Path(__file__).with_name("metrics.py")),
12:0126                         prediction_sha256=identities, source_run=str(args.run.resolve()))
12:0127             if args.out.exists() and any(args.out.iterdir()):
12:0128                 raise ValueError("Benchmark output directory must be empty")
12:0129             args.out.mkdir(parents=True, exist_ok=True)
12:0130             build_benchmark_report(args.out, result)
12:0131             return {key: value for key, value in result.items() if key not in {"details",
"prediction_sha256"}}
12:0132         if args.command == "compare":
12:0133             from .reports import compare_benchmarks
12:0134             return compare_benchmarks(args.summaries, args.out)
12:0135         if args.command == "package":
12:0136             from .reports import package_results
12:0137             return package_results(args.run, args.out)
12:0138         raise ValueError(f"Unsupported command: {args.command}")
12:0141     def add_workflow_commands(sub):
12:0142         p = sub.add_parser("scan", help="Preflight image/video inputs without loading models")
12:0143         p.add_argument("--input", type=Path, required=True)
12:0144         p.add_argument("--out", type=Path, required=True)
12:0145         p.add_argument("--recursive", action="store_true")
12:0146         p.add_argument("--hash-inputs", action="store_true")
12:0147         p.set_defaults(func=_execute)
12:0149         p = sub.add_parser("process", help="One inference pass: 2D/3D/MANO/mesh, quality checks and
resume")
12:0150         p.add_argument("--input", required=True)
12:0151         p.add_argument("--out", required=True)
12:0152         p.add_argument("--checkpoint", default=str(ROOT / "pretrained_models/wilor_final.ckpt"))
12:0153         p.add_argument("--cfg", default=str(ROOT / "pretrained_models/model_config.yaml"))
12:0154         p.add_argument("--detector", default=str(ROOT / "pretrained_models/detector.pt"))
12:0155         p.add_argument("--device", default="auto")
12:0156         p.add_argument("--detector-conf", type=probability, default=0.3)
12:0157         p.add_argument("--rescale-factor", type=positive_float, default=2.)
12:0158         p.add_argument("--batch-size", type=positive_int, default=16)
12:0159         p.add_argument("--stride", type=positive_int, default=1)
12:0160         p.add_argument("--limit", type=nonnegative_int, default=0)
12:0161         p.add_argument("--sequence-fps", type=positive_float)
12:0162         p.add_argument("--retries", type=nonnegative_int, default=1)
12:0163         for flag in ["recursive", "hash-inputs", "hash-assets", "resume", "fail-fast", "no-mesh",
"no-overlay"]:

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```

12:0164         p.add_argument("--" + flag, action="store_true")
12:0165     p.set_defaults(func=_execute)
12:0167     p = sub.add_parser("jobs", help="Inspect a run or request cancellation between frames")
12:0168     p.add_argument("--run", type=Path, required=True)
12:0169     p.add_argument("--cancel", action="store_true")
12:0170     p.set_defaults(func=_execute)
12:0172     p = sub.add_parser("mano-check", help="Validate MANO interchange or reconstruct a complete
frame")
12:0173     p.add_argument("--input", type=Path, required=True)
12:0174     p.add_argument("--out", type=Path, required=True)
12:0175     p.add_argument("--mano-dir", type=Path, default=ROOT / "mano_data")
12:0176     p.add_argument("--device", default="cpu")
12:0177     p.set_defaults(func=_execute)
12:0179     p = sub.add_parser("sequence", help="Associate hands, smooth poses, optionally fill bounded
gaps")
12:0180     p.add_argument("--run", type=Path, required=True)
12:0181     p.add_argument("--out", type=Path, required=True)
12:0182     p.add_argument("--mano-dir", type=Path, default=ROOT / "mano_data")
12:0183     p.add_argument("--max-gap", type=nonnegative_int, default=3)
12:0184     p.add_argument("--smoothing-seconds", type=positive_float, default=0.08)
12:0185     p.add_argument("--fill-gaps", action="store_true")
12:0186     p.set_defaults(func=_execute)
12:0188     p = sub.add_parser("prepare-gt", help="Adapt trusted local HOI4D pickle labels to fixed 2D GT")
12:0189     p.add_argument("--run", type=Path, required=True)
12:0190     p.add_argument("--sample-dir", type=Path, required=True)
12:0191     p.add_argument("--out", type=Path, required=True)
12:0192     p.set_defaults(func=_execute)
12:0194     p = sub.add_parser("render-sequence", help="Export offline raw/processed comparison videos")
12:0195     p.add_argument("--run", type=Path, required=True)
12:0196     p.set_defaults(func=_execute)
12:0198     p = sub.add_parser("benchmark", help="Evaluate fixed GT with one-to-one IoU matching")
12:0199     p.add_argument("--run", type=Path, required=True)
12:0200     p.add_argument("--ground-truth", type=Path, required=True)
12:0201     p.add_argument("--out", type=Path, required=True)
12:0202     p.add_argument("--variant", choices=["raw", "processed"], default="raw")
12:0203     p.add_argument("--match-iou", type=probability, default=0.1)
12:0204     p.add_argument("--pck-thresholds", nargs="+", type=positive_float, default=[5., 10., 20., 50.])
12:0205     p.set_defaults(func=_execute)
12:0207     p = sub.add_parser("compare", help="Compare benchmarks with identical evaluation protocols")
12:0208     p.add_argument("--summaries", nargs="+", type=Path, required=True)
12:0209     p.add_argument("--out", type=Path, required=True)
12:0210     p.set_defaults(func=_execute)
12:0212     p = sub.add_parser("package", help="Archive generated results with checksums, excluding
weights")
12:0213     p.add_argument("--run", type=Path, required=True)
12:0214     p.add_argument("--out", type=Path, required=True)
12:0215     p.set_defaults(func=_execute)
13:0001     """Standalone entry point for the EgoHand3D processing workflow."""
13:0002     from __future__ import annotations
13:0004     import argparse
13:0006     from .workflow_cli import add_workflow_commands
13:0009     def main(argv=None):
13:0010         parser = argparse.ArgumentParser(

```

FILE:LINE (margin) | full file mapping: source_line_map.csv

```
13:0011         prog="python -m egohand3d.workflow",
13:0012         description="Egocentric hand processing, parameter export and evaluation",
13:0013     )
13:0014     subcommands = parser.add_subparsers(dest="command", required=True)
13:0015     add_workflow_commands(subcommands)
13:0016     args = parser.parse_args(argv)
13:0017     return args.func(args)
13:0020 if __name__ == "__main__":
13:0021     raise SystemExit(main())
```